Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
дисциплины «Объектно-ориентированное программирование»
Вариант 8

Выполнил:
Наталичев Данил Андреевич
3 курс, группа ИВТ-б-о-23-2, 09.03.01 «Информатика и вычислительная техника» направленность (профиль) «Автоматизированные системы обработки информации и управления», очная форма обучения

_____
(подпись)

Руководитель практики:
Воронкин Роман Александрович, доцент департамента цифровых, роботехнических систем и электроники

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2025 г.

Тема: Элементы объектно-ориентированного программирования

Цель: приобретение навыков по работе с классами и объектами при написании программ с помощью языка Python.

Порядок выполнения работы

https://github.com/AlafurZerk/OOP_lab_1

1.Проработал примеры из методического указания.

```python
volga = River("Волга", 3530)
seine = River("Сена", 776)
nile = River("Нил", 6852)

# далее печатаем все названия рек
for river in River.all_rivers:
    print(river.name)
# output:
# Волга
# Сена
# Нил

# Класс River с методом get_info
class River:
    all_rivers = []

    def init(self, name, length):
        self.name = name
        self.length = length
        River.all_rivers.append(self)

    def get_info(self):
        print("Длина {0} равна {1} км".format(self.name, self.length))

volga = River("Волга", 3530)
seine = River("Сена", 776)
nile = River("Нил", 6852)

volga.get_info()
# Длина Волги равна 3530 км
seine.get_info()
# Длина Сены равна 776 km
nile.get_info()
# Длина Нила равна 6852 km

# Класс Pet с атрибутами класса и экземпляра
class Pet:
    kind = "mammal"
    n_pets = 0  # количество питомцев
    pet_names = []  # список имен всех питомцев
```

Рисунок 1 – Примеры

```python
class Rational:
    def init(self, a=0, b=1):
        a = int(a)
        b = int(b)

        if b == 0:
            raise ValueError("Denominator cannot be zero")

        self.numerator = abs(a)
        self.__denominator = abs(b)
        self.__reduce()

    # Сокращение дроби
    def __reduce(self):
        # Функция для нахождения наибольшего общего делителя
        def gcd(a, b):
            if a == 0:
                return b
            elif b == 0:
                return a
            elif a >= b:
                return gcd(a % b, b)
            else:
                return gcd(a, b % a)

        c = gcd(self.__numerator, self.__denominator)
        self.__numerator //= c
        self.__denominator //= c

    @property
    def numerator(self):
        return self.__numerator

    @property
    def denominator(self):
        return self.__denominator
```

Рисунок 2 - Примеры

2.Индивидуальное задание №1

Задание 1

Парой называется класс с двумя полями, которые обычно имеют имена *first* и *second*. Требуется реализовать тип данных с помощью такого класса. Во всех заданиях обязательно должны присутствовать:

- метод инициализации `__init__`; метод должен контролировать значения аргументов на корректность;
- ввод с клавиатуры `read`;
- вывод на экран `display`.

Реализовать внешнюю функцию с именем `make_тип()`, где `тип` — тип реализуемой структуры. Функция должна получать в качестве аргументов значения для полей структуры и возвращать структуру требуемого типа. При передаче ошибочных параметров следует выводить сообщение и заканчивать работу.

Номер варианта необходимо уточнить у преподавателя. В раздел программы, начинающийся после инструкции `if __name__ = '__main__':` добавить код, демонстрирующий возможности разработанного класса.

Рисунок 3 – Индивидуальное задание №1

8. Поле first — целое число, левая граница диапазона, включается в диапазон; поле second — целое число, правая граница диапазона, не включается в диапазон. Пара чисел представляет полуоткрытый интервал [first, second). Реализовать метод `rangecheck()` — проверку заданного целого числа на принадлежность диапазону.

Рисунок 4 – Вариант ИЗ №1

3.Выполнил индивидуальное задание №1.

```python
class Pair:
    def __init__(self, first, second):
        if first >= second:
            raise ValueError("Ошибка: first должен быть меньше second.")
        self.first = first
        self.second = second

    def read(self):
        self.first = int(input("Введите начало интервала (first): "))
        self.second = int(input("Введите конец интервала (second): "))
        if self.first >= self.second:
            raise ValueError("Ошибка: first должен быть меньше second.")

    def display(self):
        print(f"[{self.first}, {self.second}])")

    def rangecheck(self, number):
        return self.first <= number < self.second


def make_pair(first, second):
    try:
        return Pair(first, second)
    except ValueError as e:
        print(e)
        exit(1)


if __name__ == "__main__":
    p1 = make_pair(3, 7)
    p1.display()
    print("Проверка числа 5 в интервале:", p1.rangecheck(5))
    print("Проверка числа 7 в интервале:", p1.rangecheck(7))

    p2 = Pair(0, 1)
    try:
        p2.read()
    except ValueError as e:
        print(e)
        exit(1)
```

Рисунок 5 – Код задание №1

4.Индивидуальное задание №2.

Задание 2

Составить программу с использованием классов и объектов для решения задачи. Во всех заданиях, помимо указанных в задании операций, обязательно должны быть реализованы следующие методы:

- метод инициализации `__init__`;
- ввод с клавиатуры `read`;
- вывод на экран `display`.

Номер варианта необходимо уточнить у преподавателя. В раздел программы, начинающийся после инструкции `if __name__ = '__main__':` добавить код, демонстрирующий возможности разработанного класса.

Рисунок 6 – Задание №4

8. Создать класс Time для работы со временем в формате «час:минута:секунда». Класс должен включать в себя не менее четырех функций инициализации: числами, строкой (например, «23:59:59»), секундами и временем. Обязательными операциями являются: вычисление разницы между двумя моментами времени в секундах, сложение времени и заданного количества секунд, вычитание из времени заданного количества секунд, сравнение моментов времени, перевод в секунды, перевод в минуты (с округлением до целой минуты).

Рисунок 7 – Вариант ИЗ №2

5.Выполнил ИЗ №2.

```
class Time:
    def __init__(self, *args):
        if len(args) == 1:
            if isinstance(args[0], str):
                # Строка
                self._init_from_string(args[0])
            elif isinstance(args[0], int):
                # Секунды
                self._init_from_seconds(args[0])
            elif isinstance(args[0], Time):
                # Другой объект
                self.hours = args[0].hours
                self.minutes = args[0].minutes
                self.seconds = args[0].seconds
            else:
                raise ValueError("Invalid initialization argument")
        elif len(args) == 3:
            # Числа
            self._init_from_parts(args)
        else:
            raise ValueError("Invalid number of arguments")

        self._normalize()

    def _init_from_string(self, time_str):
        for separator in [":", " ", "-", ".", ","]:
            if separator in time_str:
                parts = list(map(int, time_str.split(separator)))
                if len(parts) == 3:
                    self._init_from_parts(parts)
                    return

        try:
            total_seconds = int(time_str)
            self._init_from_seconds(total_seconds)
        except ValueError:
            raise ValueError("Неправильный формат времени")

    def _init_from_parts(self, parts):
        self.hours, self.minutes, self.seconds = parts
```

Рисунок 8 – Код задания №2

6.Выполнил проверку black, flake8, pytest.

```
(.venv) PS C:\Users\User\PycharmProjects\OOP_lab_1> black src/ tests/
All done! ✨🍰✨
5 files left unchanged.
```

Рисунок 9 – Black

```
(.venv) PS C:\Users\User\PycharmProjects\OOP_lab_1> pytest
========================================================================================== test session starts ==========================================================================================
platform win32 -- Python 3.12.4, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\User\PycharmProjects\OOP_lab_1
configfile: pyproject.toml
collected 12 items

tests\test_pair.py ......                                                                                                                                                                                [ 50%]
tests\test_time.py ......                                                                                                                                                                                [100%]

========================================================================================== 12 passed in 0.03s ==========================================================================================
(.venv) PS C:\Users\User\PycharmProjects\OOP_lab_1>
```

Рисунок 10 - Pytest

Контрольные вопросы

1. Объявление класса в Python осуществляется с помощью ключевого слова class, за которым следует имя класса и двоеточие. Тело класса записывается с отступом.
2. Атрибуты класса принадлежат самому классу и разделяются всеми его экземплярами. Атрибуты экземпляра принадлежат конкретному объекту класса и определяются внутри методов через self.
3. Методы класса определяют поведение объектов. Они могут обрабатывать атрибуты экземпляра, выполнять операции, связанные с классом, взаимодействовать с другими объектами и предоставлять интерфейс для работы с объектами.
4. Метод init() предназначен для инициализации экземпляра класса. Он вызывается при создании нового объекта и позволяет установить начальные атрибуты экземпляра.
5. Self - это ссылка на текущий экземпляр класса, которая используется для доступа к атрибутам и методам экземпляра. Она передается первым параметром в методы экземпляра.
6. Атрибуты можно добавить в класс несколькими способами: объявить в теле класса (атрибуты класса), определить в методе init (атрибуты экземпляра) или динамически присвоить атрибут любому объекту в любой момент.

7. Управление доступом в Python осуществляется по соглашению: публичные атрибуты и методы не имеют префиксов, защищенные обозначаются одним подчеркиванием, приватные - двумя подчеркиваниями. Приватные атрибуты и методы искажаются интерпретатором, чтобы усложнить доступ извне.
8. Функция isinstance предназначена для проверки, является ли объект экземпляром указанного класса или его подкласса. Она возвращает True, если объект является экземпляром класса, иначе False.

Вывод: в ходе выполнения лабораторной работы были приобретены базовые навыки работы с ООП в языке Python.